Campus Mentorship & Peer Tutoring Platform (CampusConnect)

Comprehensive Project Report & Complete Viva Preparation Guide

> **Note for the Student:**
>
> This guide is customized specifically for your actual codebase. It explains every concept in simple, conversational English with real-world analogies so you can understand the inner workings and explain them with 100% confidence to any external examiner or teacher, without needing to memorize blind jargon.

Table of Contents

1. **Project Overview, Problem Statement, & Objectives**
2. **Technology Stack & "Why Did You Choose This?" (Viva Justifications)**
3. **System Architecture (Frontend, Backend, Data, & WebRTC)**
4. **Important Files, Functions, & Code Explained in Plain English**
5. **Complete Application Workflow (Step-by-Step Journeys)**
6. **Authentication, Data Flow, & API Request-Response Examples**
7. **Implementation Reality: Strengths, Honest Challenges, Limitations, & Improvements**
8. **40+ Likely Viva Questions & Easy, Accurate Answers**
9. **Tricky Questions Teachers Love to Ask (and How to Handle Them)**
10. **Final 5-Minute Quick-Revision Cheat Sheet**

---

1. Project Overview, Problem Statement, & Objectives

1.1 What is CampusConnect?

Campus Mentorship & Peer Tutoring Platform (CampusConnect) is a full-stack web application designed for university ecosystems. It connects junior students seeking academic help with qualified senior peers (mentors) for 1-on-1 tutoring sessions, slot-based scheduling, live interactive video classes, and structured feedback.

1.2 The Problem Statement

In most college campuses:

1. **Junior-Senior Gap:** Junior students struggle with difficult engineering courses (like Data Structures, Web Development, or Mathematics) but feel hesitant to approach professors or don't know which senior is good at what subject.
2. **Unstructured Peer Learning:** Guidance happens randomly over WhatsApp groups or library benches with no schedule tracking, no timetable conflict checks, and no accountability.
3. **No Quality Assessment:** Juniors cannot tell who is an effective tutor because there is no verified review or rating system.
4. **Scattered Meeting Tools:** Coordinating Google Meet links, WhatsApp chats, and calendar invites leads to missed sessions.

1.3 Key Objectives of the Project

- **Centralized Mentor Directory:** Allow juniors to filter mentors by department, specific subjects, and student star ratings.
- **Conflict-Free Scheduling:** Allow mentors to publish their free time slots and allow students to book them with automatic conflict detection (a student cannot book two sessions at the same time).
- **Integrated Video Classroom:** Provide an in-browser WebRTC video call with microphone/camera controls, real screen sharing, and synchronized live chat without leaving the platform.
- **Closed-Loop Feedback:** Allow students to submit a 1-to-5 star rating and written review *only after* a session is completed, automatically recalculating the mentor's cumulative rating.
- **Administrative Governance:** Provide university administrators with a high-level analytics dashboard to inspect session trends, popular subjects, and deactivate disruptive accounts.

1.4 Real Implemented Features vs What is Optional/Future

To answer viva questions honestly, know exactly what is built in your project:

- ✅ **Multi-Role Authentication:** Student, Mentor, and Admin roles with role-based route protection.
- ✅ **University Verification:** Roll number validation during signup.
- ✅ **Dynamic Mentor Directory:** Search by mentor name/bio, filter by department and subject, sort by rating and total sessions.
- ✅ **Availability & Slot Booking Engine:** Mentors create time slots; students book slots; automatic validation prevents double booking and past conflicts.
- ✅ **Real-Time Video Classroom:** PeerJS + WebRTC direct media streams with Socket.io signaling.
- ✅ **Media Controls & Screen Sharing:** Native camera/mic mute toggles, real browser screen capture (`navigator.mediaDevices.getDisplayMedia`), plus fallback animated canvas streams for testing environments where no physical webcam is plugged in.
- ✅ **In-Call Real-Time Chat:** Socket.io messaging during live video calls.
- ✅ **Rating & Review System:** Star reviews restricted strictly to completed sessions; automatic recalculation of mentor average rating.
- ✅ **Admin Management:** Visual statistics cards, subject popularity charts, user status toggle (`active` ⇄ `disabled`).

- ✅ **Dual-Mode Data Architecture:** Can run in standalone demo mode using `localStorage` simulation with initial seed data, OR switch to real Express REST API calls via `.env` configuration.

---

2. Technology Stack & "Why Did You Choose This?"

Examiners love asking: *"Why did you use React instead of plain HTML?"* or *"Why Express over Django?"* Here is the breakdown with clear viva-ready reasons:

Layer	Technology	Version	Plain-English Reason for Choosing It
Frontend Framework	React.js (with Vite)	18.2	React uses a **Component-Based Architecture** and a **Virtual DOM**, meaning only changed UI parts re-render instead of refreshing the whole page. Vite provides ultra-fast development startup using native ES modules.
Routing	React Router DOM	6.11	Enables Single Page Application (SPA) routing with protected route guards without reloading the browser.
Styling	Tailwind CSS	3.3	Utility-first CSS allows building modern, responsive, dark-mode glassmorphic interfaces fast without writing hundreds of messy separate CSS files.
Icons	Lucide React	0.244	Clean, consistent, lightweight SVG icon package.
HTTP Client	Axios	1.4	Provides cleaner syntax than `fetch()`, automatic JSON transformation, and request **interceptors** to automatically attach the JWT token to every outgoing request.
Backend Runtime	Node.js	v18+	JavaScript on the server. Allows using one single programming language (JS) across both frontend and backend. Highly efficient **event-driven, non-blocking I/O**.
Backend Framework	Express.js	4.18	Minimal and flexible web framework for Node.js. Simplifies routing, middleware creation, and JSON REST API construction.
Real-time Signaling	Socket.io	4.6	Enables bidirectional, low-latency WebSocket communication between client and server for WebRTC room coordination and live in-call text chat.
Peer-to-Peer Video	WebRTC & PeerJS	1.4	WebRTC transmits audio, video, and screen data **directly between peers** without streaming video through our server, saving server bandwidth and reducing latency. PeerJS simplifies complex WebRTC handshake code.
Security & Auth	JSON Web Tokens (JWT) & bcryptjs	9.0 / 2.4	`bcryptjs` one-way hashes passwords so plain passwords are never stored. `jsonwebtoken` creates stateless cryptographically signed tokens for session security.
Data Storage	Dual-Mode (In-Memory Store / LocalStorage + Mongoose Model blueprints)	Custom	Allows the platform to work offline/in demo mode instantly via browser `localStorage` or connect to Express in-memory `dbStore` / MongoDB.

Viva Defense Tip:

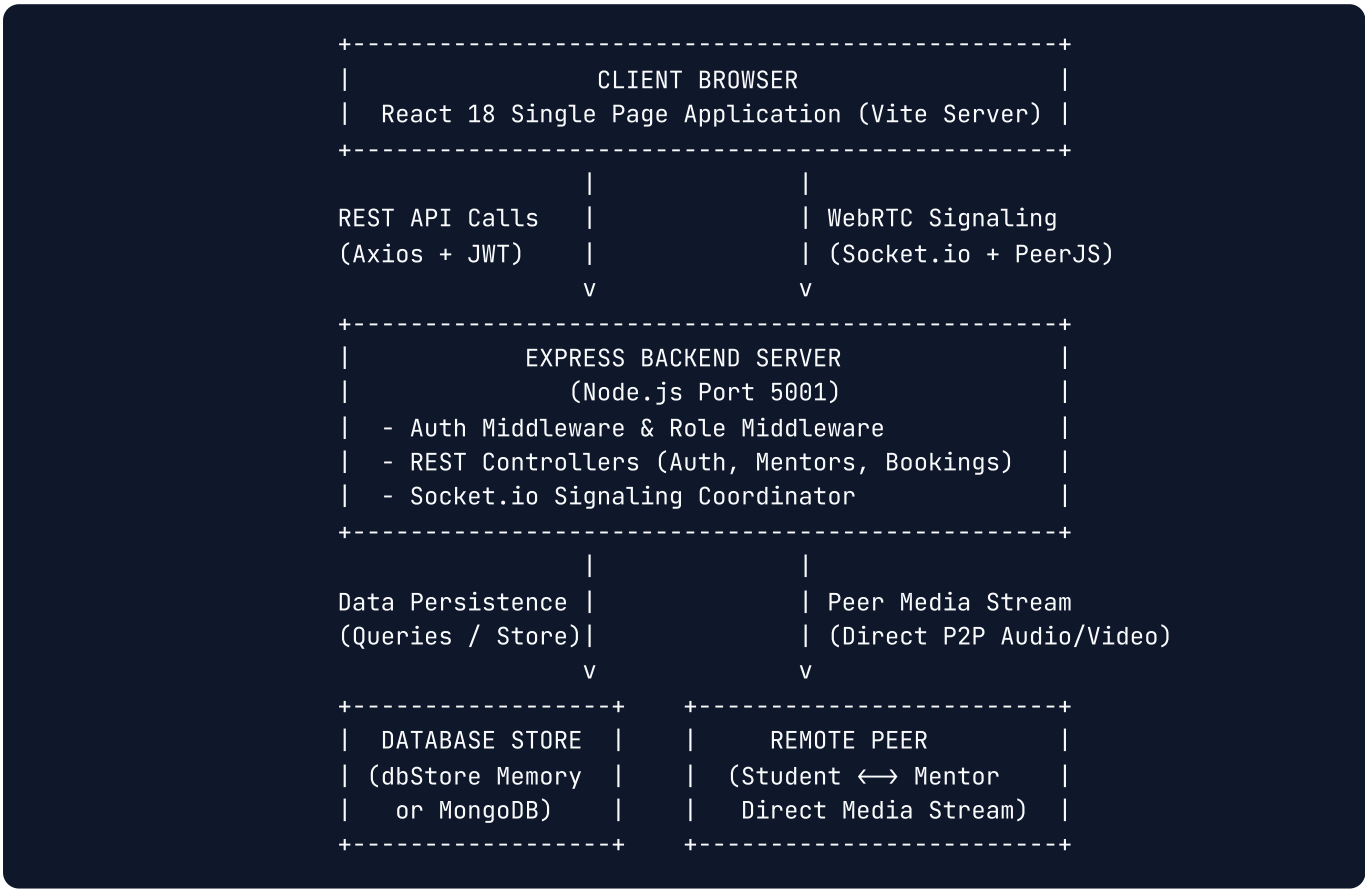
> *"Sir/Ma'am, we chose the MERN/JavaScript stack because having JavaScript on both ends (React on frontend, Node on backend) allowed us to reuse data structures and validation logic, while WebRTC and*

> *Socket.io provided zero-latency direct peer-to-peer tutoring rooms without costly third-party video subscriptions."*

---

3. System Architecture

3.1 High-Level Architectural Diagram



3.2 Architectural Flow Layers

1. **Presentation Layer (Frontend - React):**

- Components (`Navbar` , `MentorCard` , `StatCard` , `BookingModal`).
- Pages (`Home` , `Mentors` , `StudentDashboard` , `MentorDashboard` , `VideoCall` , `AdminDashboard`).
- Context State (`AuthContext` for login state, `AppContext` for bookings and mentor data, `ThemeContext` for dark mode).

1. **Service Abstraction Layer (`frontend/src/services`):**

- Contains `authService` , `mentorService` , `bookingService` , `availabilityService` , `reviewService` .
- Checks `VITE_USE_MOCK_DATA` . If true, reads/writes to `localStorage` . If false, uses `apiClient` (Axios) to call the Express server.

1. **Network & Routing Layer (Backend - Express):**

- `server.js` boots HTTP and attaches Socket.io on port 5001.

- `app.js` registers CORS, `express.json()`, and mounts sub-routers (`/api/auth`, `/api/mentors`, `/api/bookings`, etc.).

1. **Security & Middleware Layer:**

- `authMiddleware.js`: Reads `Authorization: Bearer`, verifies JWT signature, and attaches decoded user to `req.user`.
- `roleMiddleware.js`: Checks if `req.user.role` is in the allowed list (e.g. only `'student'` can book, only `'mentor'` can publish slots, only `'admin'` can view platform stats).

1. **Business Logic & Controller Layer:**

- Controllers (`authController`, `bookingController`, etc.) validate incoming inputs and call services.
- Services execute business rules (e.g., checking if a slot is already booked or if times overlap).

1. **Data Storage Layer:**

- In-memory store (`dbStore.js`) contains initialized arrays of `users`, `mentors`, `availability`, `bookings`, and `reviews`.
- Clean Object Models (`User.js`, `Mentor.js`, `Booking.js`, `Availability.js`, `Review.js`) define schema structures and contain ready-to-use Mongoose MongoDB schema blueprints.

1. **Signaling & WebRTC Layer:**

- Socket.io coordinates room joins (`join-room`), relays ICE candidates and media toggles (`media-state-change`), and broadcasts in-call chat messages (`send-message`).
- PeerJS establishes the direct peer-to-peer media connection so high-resolution video does not pass through the backend server.

---

4. Important Files, Functions, & Code Explained in Plain English

Here are the most critical files in your project. You can explain these line-by-line during your viva.

4.1 Backend Entry Points

[server.js](file:///c:/BEE%20PROJECT/Campus-Mentorship-Peer-Tutoring-Platform/backend/src/server.js)

- **Role:** The master server script. Creates the HTTP server, mounts Socket.io for WebRTC signaling, and starts listening on port 5001.
- **Key Code Concept:**

```
const server = http.createServer(app);
const io = new Server(server, { cors: { origin: env.CORS_ORIGIN } });

io.on('connection', (socket) => {
  socket.on('join-room', ({ bookingId, userId, userName }) => {
    socket.join(bookingId); // Creates a private virtual room for that booking
    socket.to(bookingId).emit('user-connected', { userId, userName, socketId: socket.id })
  });
});
```

- **Plain English:** When student and mentor open the video call for `bookingId`, Socket.io puts both into a private virtual room. When one enters, it alerts the other: *"Hey, user Arjun has entered room booking001; you can call their Peer ID now!"*

[app.js](file:///c:/BEE%20PROJECT/Campus-Mentorship-Peer-Tutoring-Platform/backend/src/app.js)

- **Role:** Express configuration. Adds JSON body parser, CORS policy, binds routes (`/api/auth`, `/api/bookings`, etc.), and adds the global error-handling middleware.

4.2 Security & Middleware

[authMiddleware.js](file:///c:/BEE%20PROJECT/Campus-Mentorship-Peer-Tutoring-Platform/backend/src/middleware/authMiddleware.js)

- **Role:** Protects private endpoints from unauthorized visitors.
- **Key Code Concept:**

```
const authHeader = req.headers.authorization;
if (!authHeader || !authHeader.startsWith('Bearer ')) {
  return res.status(401).json({ message: 'Access denied. No authentication token provided.
}
const token = authHeader.split(' ')[1];
const decoded = jwt.verify(token, env.JWT_SECRET);
req.user = decoded; // Store user details in the request object for the next function
next();
```

- **Plain English:** Like a campus security guard checking an ID badge. If you do not show a valid signed token, you are stopped with an HTTP 401 Unauthorized error. If valid, the guard writes down who you are (`req.user = decoded`) and lets you through (`next()`).

[roleMiddleware.js](file:///c:/BEE%20PROJECT/Campus-Mentorship-Peer-Tutoring-Platform/backend/src/middleware/roleMiddleware.js)

- **Role:** Enforces Role-Based Access Control (RBAC).
- **Plain English:** Even if you have an ID badge, a student cannot enter the "Faculty/Admin Only" room. If `allowedRoles.includes(req.user.role)` is false, it returns an HTTP 403 Forbidden error.

4.3 Core Business Logic (Backend Services)

[bookingService.js](file:///c:/BEE%20PROJECT/Campus-Mentorship-Peer-Tutoring-Platform/backend/src/services/bookingService.js)

- **Role:** Manages the booking lifecycle and enforces academic integrity rules.
- **Important Functions:**

1. `createBooking(studentId, { mentorId, slotId })`:

- Checks if the requested slot exists and is still `available`.
- **Timetable Conflict Check:** Searches existing bookings to see if the student already booked another session on that exact date and start time.
- Sets `slot.status = 'booked'`.
- Creates and stores a new booking object with `status: 'upcoming'`.

1. `updateBookingStatus(userId, role, bookingId, { status })`:

- Allows student, mentor, or admin to cancel or complete a session.
- **Auto-Rollback:** If a session is cancelled, it finds the mentor's slot and resets it back to `available` so another student can book it.
- **Mentor Recognition:** If a session is marked `completed`, it increments `mentor.totalSessions += 1`.

[reviewService.js](file:///c:/BEE%20PROJECT/Campus-Mentorship-Peer-Tutoring-Platform/backend/src/services/reviewService.js)

- **Role:** Handles ratings and prevents fake reviews.
- **Key Validations:**
- Booking must exist and belong to the student submitting the review.
- The booking `status` must be `'completed'` (cannot review an upcoming or cancelled session).
- Prevents duplicate reviews for the same session.
- **Dynamic Rating Recalculation:** Uses JavaScript's `.reduce()` array method:

```
const totalRating = mentorReviews.reduce((sum, r) => sum + r.rating, 0);
const avgRating = totalRating / mentorReviews.length;
mentor.rating = parseFloat(avgRating.toFixed(1));
```

4.4 Frontend State & Navigation

[AppRoutes.jsx](file:///c:/BEE%20PROJECT/Campus-Mentorship-Peer-Tutoring-Platform/frontend/src/routes/AppRoutes.jsx)

- **Role:** The client-side traffic controller. Defines all URL paths and wraps private screens inside .
- **Structure:**
- Public: `/`, `/login`, `/register`, `/mentors`, `/mentor/:id`.

- Student: `/student/dashboard`, `/student/bookings`, `/student/video/:bookingId`, `/student/profile`.
- Mentor: `/mentor/dashboard`, `/mentor/availability`, `/mentor/bookings`, `/mentor/video/:bookingId`, `/mentor/profile`.
- Admin: `/admin/dashboard`, `/admin/users`, `/admin/bookings`, `/admin/statistics`.

[AuthContext.jsx](file:///c:/BEE%20PROJECT/Campus-Mentorship-Peer-Tutoring-Platform/frontend/src/context/AuthContext.jsx)

- **Role:** Global authentication store. Uses React's Context API so any button or navbar across the entire application instantly knows who is logged in (`user`, `login`, `register`, `logout`).

[AppContext.jsx](file:///c:/BEE%20PROJECT/Campus-Mentorship-Peer-Tutoring-Platform/frontend/src/context/AppContext.jsx)

- **Role:** Global application data store. Loads mentors, user bookings, availability slots, and notifications. Exposes action functions like `createBooking`, `cancelBooking`, `completeBooking`, and `submitReview`.

4.5 The Video Classroom & WebRTC Engine

[VideoCall.jsx](file:///c:/BEE%20PROJECT/Campus-Mentorship-Peer-Tutoring-Platform/frontend/src/pages/VideoCall.jsx)

- **Role:** The real-time interactive classroom page.
- **How It Works Step-by-Step:**

1. **Hardware Acquisition:** Calls `navigator.mediaDevices.getUserMedia({ video: true, audio: true })` to capture the local webcam and mic.
2. **Fallback Simulation:** If the computer has no camera or permission is denied, it automatically triggers `createSimulatedCanvasStream()`, generating a dynamic HTML5 animated canvas stream so the user interface and call test never crash!
3. **PeerJS Registration:** Connects to PeerJS server to get a unique `peerId`.
4. **Socket Room Join:** Emits `join-room` with `{ bookingId, userId, peerId }`.
5. **Media Exchange:** When the other participant joins, an outgoing WebRTC call is initiated via `peer.call(peerId, stream)`. The remote stream is attached to the remote

   `<video>` tag.
6. **Track Replacement for Screen Share:** When the user clicks "Share Screen", `navigator.mediaDevices.getDisplayMedia()` captures the desktop screen. It replaces the outgoing

video track on the WebRTC peer connection using `videoSender.replaceTrack(screenTrack)` so the remote student instantly sees the slides or code editor.

7. **Real-time Live Chat:** Text messages typed into the chat panel are emitted via Socket.io and appended to the in-call message list.

---

5. Complete Application Workflow (Step-by-Step)

Here is the exact journey through the application from start to finish:

```
[Student Signs Up]
       |
       ▼
[Browses Mentors Page] ──(Filters by "Computer Science" & "React")──▶ [Views Mentor Profile]
                                                                              |
                                                                              ▼
[Reviews Bookings Tab] ◀──(Booking status = 'upcoming')─── [Selects Date & Books Available S
       |
       ▼ (Scheduled Time Arrives)
[Enters Video Tutoring Room (/student/video/:id)]
       |
       ├── WebRTC connects Audio/Video directly
       ├── Real Screen Sharing for code/slides
       └── Live In-Call Text Chat via Socket.io
       |
       ▼ (Class Finishes)
[Mentor Marks Session 'Completed']
       |
       ├── Mentor's total completed sessions count increments (+1)
       └── Unlocks Review Form on Student Dashboard
       |
       ▼
[Student Submits 5-Star Rating & Review] ──▶ [Mentor's Cumulative Rating Recalculates]
       |
       ▼
[Admin Dashboard] ──▶ Aggregates platform statistics, trend lines, and user controls
```

Walkthrough Scenario for Viva:

1. **Registration:** A junior student registers with Name, College Email, and University Roll Number (`22BCSE101`). Password is encrypted.
2. **Discovery:** Student clicks "Find Mentors". Mentors are displayed as cards showing their photo, department, year, star rating, and subjects. The student selects "JavaScript".
3. **Booking:** The student clicks "Book Session" on mentor Rahul Sharma's card. A modal opens showing Rahul's published slots. The student picks `August 20, 10:00 AM - 11:00 AM`. The system verifies no time clash exists, marks the slot as booked, and creates an upcoming booking.
4. **Tutoring Session:** Both student and mentor open `/student/video/:bookingId` or `/mentor/video/:bookingId`. The webcam and microphone turn on. Both can see and hear each

other, share screens, and exchange text messages in the chat panel.

5. **Session Wrap-Up:** Mentor clicks "End Call" and confirms the session is completed.
6. **Review:** Back on the Student Dashboard, an alert appears: *"You have an unreviewed session with Rahul Sharma"*. The student rates 5 stars and writes: *"Explained closures with great examples!"* Rahul's average rating updates automatically.
7. **Administration:** College admin logs in at `/admin/dashboard`. The admin sees total completed sessions, subject popularity graphs, and can suspend fraudulent accounts if needed.

---

6. Authentication, Data Operations, & API Examples

6.1 Authentication Mechanism

- **Password Security:** Uses `bcryptjs`. During registration, plain passwords pass through `bcrypt.hash(password, 10)` to produce a secure 60-character salted hash. During login, `bcrypt.compare()` verifies the entered string against the stored hash without ever decrypting it.
- **JWT (JSON Web Token):**
- **Header:** Algorithm used (`HS256`).
- **Payload:** User identification data `{ id, email, role }`.
- **Signature:** Hash generated using `JWT_SECRET` key on the server.
- **Client Storage:** The frontend receives the JWT upon login and saves it in `localStorage` (`cc_auth_token`). Axios request interceptor attaches this token to every outgoing header:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

6.2 Key API Endpoints & Request/Response Examples

1. User Login

- **Endpoint:** `POST /api/auth/login`
- **Request Body:**

```
{
  "email": "student@campusconnect.com",
  "password": "student123"
}
```

- **Response (200 OK):**

```
{
  "success": true,
  "data": {
    "token": "eyJhbGciOiJIUzI1Ni...",
    "user": {
      "id": "student_demo",
      "name": "Arjun Mehta",
      "email": "student@campusconnect.com",
      "rollNumber": "22BCSE101",
      "role": "student",
      "department": "Computer Science",
      "year": 2,
      "avatar": "https://images.unsplash.com/...",
      "mentorId": null
    }
  }
}
```


2. Get Filtered Mentors List

- **Endpoint:** `GET /api/mentors?department=Computer%20Science&subject=React&sortBy=rating`
- **Response (200 OK):**


```
{
  "success": true,
  "count": 1,
  "data": [
    {
      "id": "mentor001",
      "name": "Rahul Sharma",
      "department": "Computer Science",
      "year": 4,
      "subjects": ["JavaScript", "React", "Data Structures"],
      "rating": 4.8,
      "totalSessions": 124
    }
  ]
}
```


3. Create a Booking

- **Endpoint:** `POST /api/bookings`
- **Headers:** `Authorization: Bearer`
- **Request Body:**


```
{
  "mentorId": "mentor001",
  "slotId": "slot001"
}
```


- **Response (201 Created):**

```
{
  "success": true,
  "data": {
    "id": "booking_1693001200000",
    "studentId": "student_demo",
    "mentorId": "mentor001",
    "mentorName": "Rahul Sharma",
    "subject": "JavaScript",
    "date": "2026-08-20",
    "startTime": "10:00 AM",
    "endTime": "11:00 AM",
    "status": "upcoming"
  }
}
```


4. Post a Session Review

- **Endpoint:** `POST /api/reviews`
- **Headers:** `Authorization: Bearer`
- **Request Body:**


```
{
  "bookingId": "booking001",
  "rating": 5,
  "comment": "Super helpful session on React hooks and state management!"
}
```


- **Response (201 Created):**


```
{
  "success": true,
  "message": "Review submitted successfully.",
  "data": {
    "id": "review_1693005500000",
    "rating": 5,
    "comment": "Super helpful session on React hooks and state management!"
  }
}
```


---

7. Implementation Details, Honest Challenges, Limitations, & Improvements

Examiners respect students who give realistic, honest engineering evaluations of their own project instead of claiming it is "100% bug-free and perfect".

7.1 Key Technical Challenges Faced & Solved

1. **WebRTC NAT/Firewall Traversal:**

- *Problem:* WebRTC peer connections frequently fail on campus Wi-Fi networks due to strict Symmetric NAT and institutional firewalls.
- *Solution:* Configured Google's public STUN servers (`stun:stun.l.google.com:19302`) in PeerJS options to discover public IP addresses and NAT mappings.

1. **Hardware Testing Without Physical Webcams:**

- *Problem:* While testing on developer machines or lab computers without webcams, `navigator.mediaDevices.getUserMedia` threw a `NotFoundError`, causing crashes.
- *Solution:* Engineered an intelligent fallback generator (`createSimulatedCanvasStream`) that captures a live 30 FPS animated canvas stream when hardware media is unavailable.

1. **State Synchronization on Double Booking:**

- *Problem:* Two students opening the same mentor page at the same time could try to click the same slot.
- *Solution:* Backend service verifies slot availability inside `createBooking()` before saving and atomically flips `slot.status = 'booked'`. If already booked, it throws a clear 400 error.

1. **Dynamic Rating Maintenance:**

- *Problem:* Calculating a mentor's rating on every search query would slow down the database.
- *Solution:* Recalculated and persisted the average directly onto the Mentor object whenever a new review is submitted.

7.2 Current Limitations (Honest Viva Answers)

- **Signaling Server Scale:** Currently using a single-instance Socket.io server. In an enterprise system with 50,000 students, Socket.io would need a Redis adapter to scale across multiple Node.js server processes.
- **Group Video Calls:** WebRTC mesh architecture works great for 1-on-1 peer tutoring. For a group class of 20+ students, a Selective Forwarding Unit (SFU) like Mediasoup or Jitsi would be required to prevent browser upload overload.
- **Automated Email/SMS Reminders:** Current notifications are in-app; adding Twilio or Nodemailer would allow sending email/SMS calendar reminders 15 minutes before the session starts.

7.3 Future Enhancements

- **AI Peer Matching:** An algorithm using vector embeddings to recommend mentors based on course syllabus similarity.
- **Collaborative Whiteboard:** A shared HTML5 canvas where mentor and student can draw circuit diagrams or math equations together during the call.
- **Recorded Sessions Repository:** Optional secure cloud recording for later revision before semester exams.

---

8. 40+ Likely Viva Questions with Easy, Accurate Answers

Here are categorized viva questions with short, crystal-clear answers you can deliver smoothly:

Category A: Basic & Conceptual (Questions 1 - 10)

Q1: What is the main purpose of your project?

Answer: "It is an academic web platform that connects university students with senior peer mentors for 1-on-1 tutoring, slot scheduling, in-browser video calls, and verified reviews."

Q2: What is the technology stack used?

Answer: "We used the MERN/JavaScript stack: React.js with Vite and Tailwind CSS on the frontend, Node.js and Express.js on the backend, Socket.io and PeerJS/WebRTC for real-time video calls, and JWT with bcrypt for authentication."

Q3: What are the different user roles in your system?

Answer: "There are three distinct roles: Student (can browse mentors, book slots, attend calls, submit reviews), Mentor (can define availability slots, conduct tutoring, track reviews), and Admin (can monitor platform metrics and disable accounts)."

Q4: What is a Single Page Application (SPA)?

Answer: "An SPA is a web application that loads a single HTML file once. When the user navigates between pages, JavaScript dynamically updates the content without reloading the whole page from the server, making it fast and smooth."

Q5: What is the difference between Frontend and Backend?

Answer: "Frontend is the client-side user interface that runs in the browser and handles user interactions. Backend is the server-side logic that handles database operations, authentication, business validation, and API routing."

Q6: What is a REST API?

Answer: "REST stands for Representational State Transfer. It is an architectural style where client and server exchange data using standard HTTP methods like GET to retrieve data, POST to create data, PUT/PATCH to update data, and DELETE to remove data."

Q7: Why do we use JSON in web applications?

Answer: "JSON (JavaScript Object Notation) is a lightweight, human-readable, language-independent text format that is natively supported by JavaScript, making it ideal for client-server communication."

Q8: What is the role of `package.json`?

Answer: "It is the project manifest file. It lists project metadata, custom execution scripts (like `npm start` and `npm run dev`), and all external dependencies and libraries needed to run the project."

Q9: What is the purpose of `.env` files?

Answer: "`.env` files store environment-specific configurations and sensitive keys (like port numbers, database URLs, and JWT secrets) outside the source code, preventing security leaks when pushing

code to Git."

Q10: What is CORS and why is it needed?

Answer: "CORS stands for Cross-Origin Resource Sharing. Browsers block requests from one domain (frontend port 5173) to another domain (backend port 5001) for security. The backend uses the `cors` package to explicitly allow our frontend origin."

Category B: Frontend & React.js (Questions 11 - 20)

Q11: Why did you use React instead of Vanilla HTML/CSS/JavaScript?

Answer: "React offers reusable components, declarative syntax, and a Virtual DOM that only updates the specific parts of the screen that changed, resulting in faster rendering and much cleaner code organization."

Q12: What is the Virtual DOM and how does it work?

Answer: "The Virtual DOM is a lightweight in-memory representation of the real DOM. When data changes, React creates a new Virtual DOM tree, compares it with the old one (diffing algorithm), and efficiently updates only the changed elements in the real DOM (reconciliation)."

Q13: What are React Hooks? Which ones did you use?

Answer: "Hooks are functions that let functional components use state and lifecycle features. In our project, we heavily used `useState` for local component state, `useEffect` for fetching data and lifecycle hooks, `useContext` for global auth and app state, and `useRef` for holding video DOM elements."

Q14: What is the React Context API and why did you use it?

Answer: "Context API provides a way to pass data through the component tree without manually passing props down at every level (prop drilling). We used `AuthContext` to share user login status across all pages and `AppContext` to share bookings and mentor data."

Q15: What is the difference between `props` and `state`?

Answer: "`props` (properties) are read-only inputs passed from a parent component to a child component. `state` is internal data managed within the component that can change over time based on user interactions."

Q16: How does routing work in your application?

Answer: "We use `react-router-dom` v6 with ``, ``, and ``. It intercepts URL changes in the browser address bar and renders the corresponding page component without a full page reload."

Q17: How did you implement Protected Routes?

Answer: "We created a `` wrapper component. It checks `AuthContext`: if the user is not logged in, it redirects to `/login`. If the user's role is not in the `allowedRoles` array, it redirects them away to prevent unauthorized access."

Q18: What is Tailwind CSS and what is its advantage?

Answer: "Tailwind is a utility-first CSS framework. Instead of writing custom class names in separate CSS files, we compose pre-built utility classes directly in JSX (`flex`, `p-4`, `rounded-xl`). It speeds up styling and ensures design consistency."

Q19: What is Axios and why use it over native `fetch()`?

Answer: "Axios automatically transforms JSON responses, provides cleaner error handling, and crucially supports **interceptors**, which allow us to automatically attach the JWT Bearer token to all outgoing HTTP request headers."

Q20: What happens if a user enters an invalid URL in your app?

Answer: "Our routing table has a fallback wildcard route `} />` that gracefully renders a custom 404 page with a button to return to the homepage."

Category C: Backend, Node.js, & Express (Questions 21 - 30)

Q21: What is Node.js and how does it handle concurrency?

Answer: "Node.js is an open-source, cross-platform JavaScript runtime built on Chrome's V8 engine. It uses a single-threaded, event-driven architecture with an Event Loop and non-blocking I/O to handle thousands of concurrent connections efficiently."

Q22: What is Express.js middleware? Give an example from your project.

Answer: "Middleware is a function that has access to the request (`req`), response (`res`), and the `next` function in the application's request-response cycle. An example is our `authMiddleware`, which verifies the JWT token before allowing requests to reach private controllers."

Q23: How do you handle errors in your Express backend?

Answer: "We use a centralized error middleware (`errorMiddleware.js`) registered at the very end of `app.js`. If any route controller encounters an error, it passes it to `next(error)`, which formats a standard JSON response with status 500 and the error message."

Q24: What is the role of `bcryptjs` in your project?

Answer: "`bcryptjs` provides one-way salted password hashing. When a user registers, we hash their password with a salt round of 10. Salt adds random characters before hashing, preventing dictionary attacks and rainbow table lookups."

Q25: What is JWT and how does authentication work with it?

Answer: "JWT (JSON Web Token) is a stateless authentication method. On successful login, the server signs a token containing the user ID and role with a secret key. The frontend stores this token and sends it in the `Authorization` header on subsequent requests. The server verifies the token's digital signature without querying the database for session state."

Q26: What are the three parts of a JSON Web Token?

Answer: "1. **Header:** specifies token type and hashing algorithm (e.g. HS256). 2. **Payload:** contains user claims (id, email, role). 3. **Signature:** cryptographic hash of header + payload created using the

secret key."

Q27: How does your backend prevent duplicate availability slots?

Answer: "In `availabilityService.js`, before adding a slot, we search existing slots for that mentor. If a slot matches the same mentor, same date, and same start time, we reject it with an error: *'An availability slot already exists for this date and start time.'*"

Q28: How does your backend prevent a student from double booking themselves?

Answer: "In `bookingService.js`, when a student attempts to book a slot, we check their existing bookings with status `upcoming`. If they already have a session on that same date and time, the booking is rejected."

Q29: What happens when a booking is cancelled?

Answer: "The booking status changes to `'cancelled'`. The backend automatically finds the mentor's corresponding availability slot and resets its status from `'booked'` back to `'available'` so other students can book it."

Q30: How is a mentor's rating calculated?

Answer: "When a student submits a review, the backend fetches all reviews for that mentor, uses `.reduce()` to calculate the sum of ratings, divides by the total count, rounds to 1 decimal place, and saves the updated score on the mentor record."

Category D: WebRTC & Real-Time Communication (Questions 31 - 40)

Q31: What is WebRTC?

Answer: "WebRTC (Web Real-Time Communication) is an open-source standard and browser API that allows peer-to-peer transmission of audio, video, and data directly between browsers without routing media streams through a server."

Q32: What is the difference between Socket.io and WebRTC in your project?

Answer: "Socket.io acts as the **Signaling Server** (it introduces the two browsers, exchanges connection information, and handles text chat). WebRTC handles the **actual media streaming** (voice, webcam video, and screen sharing) directly between the two computers."

Q33: What is Signaling in WebRTC?

Answer: "Before two browsers can stream video directly, they need to know each other's IP address, ports, and supported video formats. Signaling is the discovery process where browsers exchange this information (SDP offers/answers and ICE candidates) via a server (Socket.io)."

Q34: What is a STUN server and why did you use Google's STUN?

Answer: "STUN stands for Session Traversal Utilities for NAT. Most user devices are behind home or college Wi-Fi routers (NAT) and have private IP addresses. A STUN server tells the browser its public-facing IP address and port so the remote peer can connect to it."

Q35: What is PeerJS and why did you use it?

Answer: "PeerJS is a library that wraps WebRTC's complex connection setup (`RTCPeerConnection`, ICE handling, and offer/answer negotiations) into simple, clean events like `peer.call(id, stream)` and `peer.on('call', ...)`."

Q36: How does Screen Sharing work in your application?

Answer: "We use the browser API `navigator.mediaDevices.getDisplayMedia({ video: true })` to capture the user's screen. We then use WebRTC's `replaceTrack()` method to swap the webcam video track with the screen track on the live connection."

Q37: What is the Canvas Stream Fallback you implemented?

Answer: "If a device has no physical camera or the user blocks camera permission, `getUserMedia` would normally throw an error. We built a fallback that creates an HTML5

`<canvas>`, animates a live pulsing gradient avatar, and captures it as a 30 FPS MediaStream using `canvas.captureStream(30)` so the video room never breaks."

Q38: How does the in-call chat work?

Answer: "The chat panel uses Socket.io. When a user submits a message, the client emits `send-message` to the server with the `bookingId`. The server relays it via `socket.to(bookingId).emit('receive-message')` to all participants in that room."

Q39: How do microphone mute and camera toggle work during the call?

Answer: "Each `MediaStream` contains audio and video tracks. When the user clicks mute, we set `track.enabled = false`. This stops audio transmission while keeping the WebRTC peer connection open. We also emit a socket event so the remote peer sees a 'Microphone Muted' indicator."

Q40: Why not send all video streams through your Node.js server?

Answer: "Video streams require huge network bandwidth and CPU processing. If 50 students streamed 1080p video through a single Node.js server, the server would crash. WebRTC solves this by offloading the video traffic directly between the client devices."

---

9. Tricky "Examiner Favorite" Questions & How to Defend Your Project

These are the difficult, curveball questions examiners ask to test whether you actually built the project or just downloaded a template:

Tricky Question 1:

> *"I noticed you have a dual-mode system with `localStorage` and Express. Why did you implement this instead of just connecting directly to a database?"*

- **Best Viva Answer:**

"Sir/Ma'am, in real-world software engineering, having decoupled service layers is a best practice. By architecting our service modules (`authService` , `bookingService`) to check `VITE_USE_MOCK_DATA` , we achieved two major advantages: first, the application can be demonstrated immediately in academic presentations or viva environments without requiring a local database server running; second, when connecting to MongoDB or Express, our entire React UI layer remains untouched because the service interfaces are identical."

Tricky Question 2:

> *"Why did you use JWT instead of traditional server-side Sessions with Cookies?"*

- **Best Viva Answer:**

*"Server-side sessions require the backend to maintain a stateful session store (like Redis or in-memory session tables). If the backend restarts or scales across multiple instances, sessions can be lost unless synchronized. JWT is **stateless**: the user's identity is digitally signed into the token itself. The server only has to verify the mathematical signature, reducing server memory usage and making the API scalable and REST-compliant."*

Tricky Question 3:

> *"What happens if a student tries to review a mentor before the session actually takes place?"*

- **Best Viva Answer:**

"Our system strictly forbids this at two levels: in the frontend UI, the 'Review' button is conditionally rendered only for bookings with `status === 'completed'` . Furthermore, on the backend in `reviewService.js` , we validate `booking.status === 'completed'` . If someone sends a forged POST request for an upcoming booking, the backend rejects it with an HTTP error: 'Cannot review this booking. The session is not marked as completed yet.'"

Tricky Question 4:

> *"What is the difference between symmetric and asymmetric NAT, and why does WebRTC sometimes fail on university Wi-Fi?"*

- **Best Viva Answer:**

"STUN servers work by discovering public IP:Port mappings. On normal (Symmetric) routers, the mapping remains constant for outward connections. But institutional university firewalls often use Symmetric NAT, where each destination IP gets a different mapped port. In that scenario, direct P2P

connections cannot resolve through STUN alone, and an intermediary TURN relay server (Traversal Using Relays around NAT) is required to route the media packets."

Tricky Question 5:

> *"Why did you use `bcrypt` instead of SHA-256 or MD5 for password storage?"*

- **Best Viva Answer:**

"MD5 and SHA-256 are general-purpose cryptographic hash functions designed to be extremely fast. Because they are fast, hackers can use modern GPUs to calculate billions of hashes per second using brute force. `bcrypt` is specifically designed for passwords: it is intentionally slow and includes a configurable 'work factor' (salt rounds). This makes brute-force attacks computationally infeasible."

Tricky Question 6:

> *"How do you prevent SQL Injection and NoSQL Injection in your project?"*

- **Best Viva Answer:**

"First, we do not concatenate raw user input strings into queries. Second, in our Express routes, we use strict input validation (`validationMiddleware.js` and `validators.js`) to sanitize and type-cast incoming parameters. Third, our data models use strict property mapping so extra or malicious fields sent in the request body are ignored."

---

10. Final 5-Minute Quick-Revision Cheat Sheet

Keep this sheet in mind right before entering your viva room:

Core Definitions

- **React 18:** Component-based UI library with Virtual DOM for dynamic, partial page re-rendering.
- **Node.js:** Chrome V8 JavaScript runtime for scalable, non-blocking, asynchronous backend services.
- **Express.js:** Lightweight web framework for Node.js handling HTTP routes, middleware, and JSON APIs.
- **WebRTC:** Peer-to-peer browser standard for live audio/video/screen streaming without routing through a server.
- **Socket.io:** WebSocket library for real-time signaling, room coordination, and in-call chat.
- **JWT:** Compact, URL-safe, digitally signed token for stateless authentication.
- **bcrypt:** Salted, slow-hashing algorithm for secure one-way password storage.

Key HTTP Status Codes

- `200 OK`: Request succeeded.

- `201 Created`: Resource successfully created (new booking or review).
- `400 Bad Request`: Client validation error (missing roll number or timetable conflict).
- `401 Unauthorized`: Missing or invalid JWT authentication token.
- `403 Forbidden`: Authenticated user lacks permission (student trying to access admin dashboard).
- `404 Not Found`: Requested resource does not exist.
- `500 Internal Server Error`: Unhandled backend exception caught by global error middleware.

Crucial Project Ports & Routes

- **Frontend Port:** `5173` (Vite dev server)
- **Backend Port:** `5001` (Express REST & Socket.io signaling)
- **Public Routes:** `/` (Home), `/login`, `/register`, `/mentors`, `/mentor/:id`
- **Student Routes:** `/student/dashboard`, `/student/bookings`, `/student/video/:bookingId`
- **Mentor Routes:** `/mentor/dashboard`, `/mentor/availability`, `/mentor/bookings`, `/mentor/video/:bookingId`
- **Admin Routes:** `/admin/dashboard`, `/admin/users`, `/admin/bookings`, `/admin/statistics`

Master Formula: The 30-Second Elevator Pitch

> *"Our project, CampusConnect, is a full-stack campus mentorship platform built with React, Node.js, Express, Socket.io, and WebRTC. It solves unstructured peer guidance by enabling university juniors to search mentors by subject and rating, book conflict-free time slots, attend 1-on-1 interactive video tutoring sessions with screen sharing, and leave verified ratings that automatically calculate mentor credibility, all protected by JWT role-based security."*

Report generated from project source code analysis.